折腾笔记[20]-使用rust交换protobuf数据

摘要

使用rust和protobuf3交换数字数据.

关键词

rust;protobuf;

关键信息

项目地址:
[https://github.com/ByeIO/bye.rosette.rs/blob/rust/crates/rosette_core/examples/protobuf_msg/protobuf_msg.rs]

```toml
[dependencies]
# protobuf解析
prost = { version = "0.13.5", features = ["prost-derive"] }
# gRPC通信
tonic = { version = "0.13.0", features = ["prost"] }

# build专用依赖
[build-dependencies]
tonic-build = "0.8.0"
```

原理简介

protobuf3简介

[https://protobuf.com.cn/programming-guides/proto3/]
[https://protobuf.com.cn/overview/]
Protocol buffers 是一种语言中立、平台中立、可扩展的结构化数据序列化机制。
它类似于 JSON，但更小、更快，并且可以生成原生语言绑定。您可以定义一次数据结构，然后使用特殊的生成源代码，轻松地将结构化数据写入和读取到各种数据流，并使用各种语言。

Protocol buffers 是定义语言（在 .proto 文件中创建）、proto 编译器生成的用于与数据交互的代码、特定于语言的运行时库、写入文件（或通过网络连接发送）的数据的序列化格式以及序列化数据的组合。

Protocol buffers 为类型化、结构化数据包提供了一种序列化格式，这些数据包的大小可达几兆字节。该格式既适用于临时网络流量，也适用于长期数据存储。Protocol buffers 可以扩展新信息，而不会使现有数据失效或需要更新代码。

Protocol buffers 是 Google 最常用的数据格式。它们广泛用于服务器间通信以及磁盘上数据的归档存储。
Protocol buffer 消息 和 服务 由工程师编写的 .proto 文件描述。

DDS对比gRPC

gRPC和DDS（数据分发服务）都是用于分布式系统通信的技术，但它们在架构、通信模型、性能和适用场景等方面存在显著差异。以下是两者的详细对比：

通信模型

- **gRPC**：基于客户端-服务器架构，采用请求-响应模式。客户端发起请求，服务器处理请求并返回响应。
- **DDS**：基于发布-订阅模型，采用点对点的对等网络架构。每个节点既可以发布数据，也可以订阅数据，无需中心服务器。

性能

- **gRPC**：
  - 使用HTTP/2协议，支持多路复用、持久连接和高效的二进制数据传输，性能优于传统的REST。
  - 在处理小数据量时性能表现优异，但在传输大二进制数据时，单次传输（Unary）方式可能不如流式传输（Streaming）。
- **DDS**：
  - 性能与“原生”RPC实现相当，在某些场景下甚至比Web服务快数十倍。
  - 由于其对等架构，数据传输路径更短，适合低延迟和高吞吐量的场景。

可靠性和容错性

- **gRPC**：由于依赖中心服务器，可能存在单点故障风险。不过，通过负载均衡和冗余部署可以缓解这一问题。
- **DDS**：对等架构使其具有更好的容错性。即使部分节点失效，其他节点仍可继续通信。

开发复杂度

- **gRPC**：需要定义接口并生成代码，开发和维护相对复杂。但其强类型接口和代码生成工具可以减少错误。
- **DDS**：支持自动代码生成，开发过程较为简化。

适用场景

- **gRPC**：
  - 适用于对性能要求较高的内部服务通信，如实时应用、微服务架构中的服务间通信。
  - 适合需要强类型接口和自动代码生成的场景。
- **DDS**：
  - 广泛应用于对可靠性、低延迟和高吞吐量要求极高的领域，如航空航天、工业自动化和物联网。
  - 适合需要高度解耦和动态通信的场景。

总结

- 如果你的系统需要高性能、低延迟的内部通信，且对开发复杂度和维护成本可以接受，可以选择**gRPC**。
- 如果你的系统需要高度解耦、动态通信和容错能力，且对性能要求极高，可以选择**DDS**。

实现

[https://rust-book.junmajinlong.com/ch101/02_Protobuf_tonic.html]

build.rs

```rust
#![allow(unused)]

//! 预处理protobuf文件为rust文件

// 标准库
use std::io::Result;

// 主函数
fn main() -> Result<()> {
    // 转换proto文件为rust文件
    tonic_build::configure()
            // 是否编译生成用于服务端的代码
            .build_server(true)
            // 是否编译生成用于客户端的代码
            .build_client(true)
            // 输出的路径，此处指定为项目根目录下的assets目录
            .out_dir("assets")
            // 指定要编译的proto文件路径列表，第二个参数是提供protobuf的扩展路径，
            // 因为protobuf官方提供了一些扩展功能，自己也可能会写一些扩展功能，
            // 如存在，则指定扩展文件路径，如果没有，则指定为proto文件所在目录即可
            .compile(&["./assets/num.proto"], &["protos"])?;
    Ok(())
}
```

main.rs

```rust
#![allow(unused)]

//! 使用protobuf解析消息文件num.proto为rust代码并测试相互通信

// 导入proto生成的rust文件
pub mod num {
    include!("../../assets/num.rs");
}

use num::Number;
use prost::Message;

fn main() {
    // 构造一个数字消息
    let number = Number { num: 42 };

    // 将数字消息序列化为字节数组
    let serialized = serialize_number(&number).unwrap();
    println!("序列化后的字节数组: {:?}", serialized);

    // 将字节数组反序列化为数字消息
    let deserialized = deserialize_number(&serialized).unwrap();
    println!("反序列化后的数字消息: {:?}", deserialized.num);
}

/// 将Number消息序列化为字节数组
///
/// 参数:
/// - number: 要序列化的Number消息
///
/// 返回:
/// - 序列化后的字节数组，如果失败则返回错误
fn serialize_number(number: &Number) -> Result<Vec<u8>, prost::EncodeError> {
    // 使用prost的Message trait的encode方法进行序列化
    let mut buf = Vec::new();
    number.encode(&mut buf)?;
    Ok(buf)
}

/// 将字节数组反序列化为Number消息
///
/// 参数:
/// - buf: 要反序列化的字节数组
///
/// 返回:
/// - 反序列化后的Number消息，如果失败则返回错误
fn deserialize_number(buf: &[u8]) -> Result<Number, prost::DecodeError> {
    // 使用prost的Message trait的decode方法进行反序列化
    Number::decode(buf)
}
```

num.rs

```rust
/// 数字消息
#[allow(clippy::derive_partial_eq_without_eq)]
#[derive(Clone, PartialEq, ::prost::Message)]
pub struct Number {
    #[prost(int64, tag = "1")]
    pub num: i64,
}
```

num.proto

PROTO 复制 全屏

```proto
syntax = "proto3";

// 数字消息
message Number {
  int64 num = 1;
}
```

效果

```
序列化后的字节数组: [8, 42]
反序列化后的数字消息: 42
```

分类: 折腾笔记
标签: Rust

好文要顶 关注我 收藏该文 微信分享

qsBye
粉丝 - 27 关注 - 0
会员号: 107

0 推荐 0 反对

« 上一篇: 折腾笔记[19]-使用rust发布和订阅ros2消息

posted @ 2025-04-07 01:23 qsBye 阅读(0) 评论(0) MD 编辑 收藏 举报

刷新评论 刷新页面 返回顶部

发表评论

编辑 预览
支持 Markdown
自动补全
提交评论 退出
[Ctrl+Enter快捷键提交]

【推荐】100%开源！大型工业跨平台软件C++源码提供，建模，组态！
【推荐】还在用 ECharts 开发大屏？试试这款永久免费的开源 BI 工具！
【推荐】国内首个AI IDE，深度理解中文开发场景，立即下载体验Trae
【推荐】编程新体验，更懂你的AI，立即体验豆包MarsCode编程助手
【推荐】轻量又高性能的 SSH 工具 IShell：AI 加持，快人一步

编辑推荐：
· 探秘 MySQL 索引底层原理，解锁数据库优化的关键密码(下)
· 大模型 Token 究竟是啥：图解大模型Token
· 35岁程序员的中年求职记：四次碰壁后的深度反思
· 继承的思维：从思维模式到架构设计的深度解析
· 如何在 .NET 中 使用 ANTLR4

阅读排行：
· BotSharp 5.0 MCP：迈向更开放的AI Agent框架
· 分享 3 款基于 .NET 开源且免费的远程桌面工具
· 在线聊天系统中的多窗口数据同步技术解密
· 2025，回顾出走的 10 年
· 设计模式脉络

公告
昵称：qsBye
园龄：8年5个月
粉丝：27
关注：0

2025年4月

搜索

常用链接
我的随笔
我的评论
我的参与
最新评论
我的标签

我的标签
Rust(16)
SLAM(5)
线性代数(2)
LLM(1)

随笔分类
51单片机(10)
Android开发(18)
bouffalo单片机(1)
esp32单片机(24)
GNU Radio(4)
LLM精彩问答(7)
MATLAB(6)
RaspberryPi单片机(6)
SCILAB(3)
stm32单片机(16)
TI单片机(3)
WebAssembly(3)
翻译(5)
高云FPGA(3)
公众号(2)
鸿蒙开发(3)
机器人(13)
前端开发(12)
嵌入式Linux(3)
人工智能(4)
运维相关(16)
折腾笔记(22)

随笔档案
2025年4月(3)
2025年3月(4)
2025年2月(6)
2025年1月(11)
2024年12月(1)
2024年10月(1)
2024年9月(4)
2024年8月(2)
2024年6月(3)
2024年5月(9)
2024年4月(3)
2024年3月(10)
2024年2月(8)
2024年1月(9)
2023年12月(4)
2023年11月(6)
2023年10月(4)
2023年9月(4)
2023年8月(10)
2023年7月(16)
2023年6月(2)
2023年5月(2)
2023年4月(3)
2023年3月(2)
2023年2月(7)
2023年1月(17)
2022年12月(11)
2022年11月(14)
2022年10月(8)
2022年9月(17)
更多

阅读排行榜
1. OpenWrt安装Filebrowser网页文件管理器(11674)
2. esp32笔记[5]-基于I2S协议实现音频播放(9571)
3. 谈谈NAS的硬盘SATA背板(6749)
4. 三星平板的笔记模板(比例:16:10)(4329)
5. 电犀牛R66s软路由体验(4283)

评论排行榜
1. 51单片机笔记[1]-基础实验(1)
2. zemax光学设计笔记[5]-ZPL或ZOS-API的简单使用(1)
3. 小米6安装Ubuntu Touch(1)

推荐排行榜
1. 折腾笔记[15]-配置Zed编辑器(1)
2. esp32笔记[5]-基于I2S协议实现音频播放(1)
3. esp32笔记[1]-blink闪灯(1)
4. stm32笔记[2]-HAL库驱动2.4寸屏幕(1)
5. 基于esp32的MQTT开关门检测模块(1)

最新评论
1. Re:zemax光学设计笔记[5]-ZPL或ZOS-API的简单使用
请问博主用vsstudio运行的时候无法读取类型库文件(ZOSAPI.tlb)如何解决啊，ZOSAPII.tlb能用import读取，ZOSAPI.tlb读取却报错，研究了一天了。这两个文件在zosa...
--赤Xin
2. Re:小米6安装Ubuntu Touch
您好！请问应该如何扩容docker呢?
我按照本文给出的方案安装之后docker容量只有4G
--VantHoff
3. Re:51单片机笔记[1]-基础实验
大学买的单片机就没打开过.
--mossxzzom

Copyright © 2025 qsBye
Powered by .NET 9.0 on Kubernetes